

Claude Code CLI – Vyčerpávající příručka

Komplexní průvodce použitím, konfigurací, rozšířeními a osvědčenými postupy

Verze dokumentu: 1.0 **Datum:** 10. května 2026 **Cílová skupina:** Vývojáři, DevOps, technická publika **Pokrytí:** Všechny operační systémy (Windows, macOS, Linux) **Předpokládá se:** Claude Code je již nainstalovaný a přihlášený

Obsah

1. [Úvod a kontext](#)
2. [Architektura a klíčové koncepty](#)
3. [Spouštění a CLI parametry](#)
4. [Subcommands \(`claude config` , `claude mcp` , ...\)](#)
5. [Vestavěné slash příkazy](#)
6. [Vlastní slash příkazy](#)
7. [Konfigurační hierarchie a `settings.json`](#)
8. [Paměť: `CLAUDE.md` a importy](#)
9. [Environment proměnné](#)
10. [Permissions a režimy oprávnění](#)
11. [Vestavěné nástroje \(Tools\)](#)
12. [Subagenti](#)
13. [Skills](#)
14. [Hooks](#)
15. [MCP \(Model Context Protocol\)](#)
16. [Plugins a marketplaces](#)
17. [IDE integrace \(VS Code, JetBrains\)](#)
18. [Headless režim, CI/CD a automatizace](#)
19. [Stream-JSON formát](#)
20. [Output styles a vzhled](#)
21. [Vim mode, klávesové zkratky a terminál](#)
22. [Cost a usage tracking](#)
23. [Best practices](#)
24. [Troubleshooting](#)
25. [Příloha A – Kompletní reference flagů](#)
26. [Příloha B – Reference klíčů `settings.json`](#)
27. [Příloha C – Reference environment proměnných](#)
28. [Příloha D – Frontmatter cheat sheet \(agent / skill / command\)](#)
29. [Příloha E – Glossář pojmů](#)
30. [Příloha F – Užitečné zdroje a odkazy](#)

1. Úvod a kontext

Claude Code je oficiální CLI nástroj společnosti Anthropic určený pro „agentní“ programování v terminálu. Na rozdíl od chatovacích nástrojů Claude Code aktivně čte a upravuje váš kód, spouští příkazy v shellu, prochází repozitář, používá

nástroje (file ops, web search, MCP) a koordinuje práci napříč více soubory. Cílem je posunout interakci s LLM z „odpovídání na otázky“ do „spolupráce na úkolu“.

1.1. Pro koho je tato příručka

Tato příručka je psána s předpokladem, že čtenář:

- Pracuje s terminálem na denní bázi a rozumí rozdílu mezi PowerShellem, cmd, bash a zsh.
- Zná principy verzovacích systémů (Git) a běžně pracuje s konfiguračními soubory v JSON / YAML.
- Chce z Claude Code vytěžit maximum: nejen spouštět dotazy, ale i konfigurovat, automatizovat a integrovat ho do svého pracovního postupu (CI/CD, IDE, vlastní hooks, MCP servery).

Na úrovni příkladů jsou pokryty Windows (PowerShell, cmd, WSL), macOS i Linux. Pokud existuje rozdíl mezi platformami, bude na něj upozorněno.

1.2. Co příručka pokrývá a co ne

Pokrývá:

- Veškeré CLI parametry, subcommandy, slash příkazy, konfigurace, oprávnění, environment proměnné.
- Skills, subagenty, hooks, MCP servery a pluginy.
- Headless režim, CI/CD pipeline integrace, stream-JSON.
- Best practices, troubleshooting, ukázkové konfigurace pro reálné scénáře.

Nepokrývá:

- Instalaci a první spuštění (autentizaci, registraci) – ty jsou v oficiální dokumentaci.
- Internals modelu (Claude jako LLM, prompt engineering od základů).

1.3. Konvence v textu

V příručce platí tyto typografické konvence:

- **příkaz** – cokoliv, co se zadává do shellu nebo souboru.
- **--flag** – konkrétní CLI parametr.
- **~/ .claude/** – domovský adresář Claude Code (na Windows obvykle **%USERPROFILE%\ .claude**).
- **\$ENV_VAR** – environment proměnná. Na Windows v PowerShellu se použije **\$env:ENV_VAR**.
-  – upozornění na bezpečnostní implikace.
-  – pozor, snadno přehlédnutelná past.
-  – praktický tip.

2. Architektura a klíčové koncepty

Předtím, než se pustíme do flagů, je užitečné mít mentální model toho, co se vlastně děje.

2.1. Co je relace (session)

Když spustíte **claude** v adresáři, vznikne **relace**. Relace má:

- **Working directory** (a volitelně přidané další adresáře přes **--add-dir**).

- **Konverzační historii** – tu zapisuje a přepisuje Claude i uživatel.
- **Načtenou paměť** (CLAUDE.md soubory napříč scope hierarchií).
- **Sadu povolených nástrojů** (allowed/disallowed tools).
- **Permission mode** – jak agresivně potvrzovat akce.
- **Sadu MCP serverů** + jejich autorizace.
- **Session ID** – identifikátor (UUID), který se uloží do transkriptu na disk a kterým lze relaci později obnovit.

Transkripty se ukládají v `~/ .claude/projects/<projekt>/` a typicky se rotují podle `cleanupPeriodDays` v `settings.json`.

2.2. Hierarchie konfigurace

Claude Code drží konfiguraci na čtyřech úrovních:

Úroveň	Lokace	Čte se	Commitovat?
Managed	systémově řízeno (Enterprise)	nejvyšší priorita	ne (řízené IT)
Local	<code>.claude/settings.local.json</code>	nepřepisuje project	ne (gitignore)
Project	<code>.claude/settings.json</code>	sdíleno týmem	ano
User	<code>~/ .claude/settings.json</code>	globálně přes všechny projekty	ne

Při konfliktu vyhrává vyšší úroveň (managed > local > project > user).

2.3. Tři mechanismy rozšiřitelnosti

Claude Code lze rozšířit třemi navzájem komplementárními způsoby:

- **Slash commands** – krátké uživatelské makro, vždy spuštěné explicitně (`/název`).
- **Subagents** – izolované „mini-Claudy“, které lze pověřit specifickým úkolem; mají vlastní system prompt a sadu nástrojů.
- **Skills** – instruktážní balíčky s progressive disclosure: Claude o nich ví průběžně a zapojí je sám, když uvidí relevantní úkol.

K tomu se ještě přidávají **Hooks** (deterministické skripty napojené na životní cyklus) a **MCP servery** (externí nástroje a zdroje dat).

2.4. „Pravidlo nejmenšího překvapení“ pro Claude Code

Při psaní konfigurace, agentů a skillů držte tři principy:

- *Budte konkrétní v `description` polích.* Claude se podle nich rozhoduje, kdy něco použít.
- *Omezujte nástroje na minimum.* Agent na refaktoring nepotřebuje WebSearch.
- *Pište memo, ne romány.* `CLAUDE.md` se čte při každém startu sezení a žere kontext.

3. Spouštění a CLI parametry

Základní volání:

```
claude # interaktivní REPL v aktuálním adresáři
claude "Vysvětli mi src/auth.cs" # interaktivní s úvodním promptem
claude -p "Zhrň soubor README" # headless – vrátí výstup a skončí
```

Následují všechny relevantní flagy seskupené podle účelu.

3.1. Headless / one-shot režim

Flag	Význam
-p, --print	Spustí dotaz neinteraktivně, výstup pošle na stdout a skončí. Nutný pro CI/CD.
--output-format	Formát stdout: text (výchozí), json (jediný JSON s metadaty), stream-json (NDJSON proud událostí).
--input-format	Formát stdin: text (výchozí) nebo stream-json (NDJSON s user zprávami).
--max-turns N	Tvrdý limit počtu turnů (kolikrát Claude smí odpovědět).
--verbose	Detailní log; <i>vyžadováno</i> pro stream-json output, jinak nedostanete delta tokeny.

Příklad – jednorázový code review v CI:

```
claude -p "Najdi v této PR možné race conditions a vrať seznam v markdownu" \
--output-format json \
--max-turns 12 \
--permission-mode plan > review.json
```

Příklad – streamování událostí pro live UI:

```
claude -p "Vygeneruj unit testy pro UserService" \
--output-format stream-json \
--verbose \
| jq -r 'select(.event?.delta?.text?) | .event.delta.text'
```

3.2. Pokračování a obnovování relace

Flag	Význam
-c, --continue	Načte poslední relaci v aktuálním adresáři.
-r SESSION_ID, --resume SESSION_ID	Obnoví konkrétní relaci podle UUID.
--session-id LABEL	Přiřadí relaci textovou značku (používá se v některých automatizovaných tocích pro snadnou identifikaci).

```

claude --continue # navaž tam, kde jsi přestal
claude --resume 4f6c2d11-7c5a-4b8e-9b1d-7c1e0f5a2b30

```

💡 V automatizaci si můžete uložit `session_id` z `--output-format json` a později ho obnovit:

```

SESSION=$(claude -p "Začni refaktor" --output-format json | jq -r '.session_id')
# ... později ...
claude -p "Pokračuj refaktorem AuthService" --resume "$SESSION"

```

3.3. Volba modelu

Flag	Význam
<code>--model MODEL_NAME</code>	Explicitní model pro relaci, např. <code>claude-opus-4-6</code> , <code>claude-sonnet-4-6</code> , <code>claude-haiku-4-5</code> .
<code>--fallback-model MODEL_NAME</code>	Záložní model, pokud primární není dostupný (rate limit, výpadek).

```

claude --model claude-opus-4-6 --fallback-model claude-sonnet-4-6

```

3.4. Adresáře a pracovní rozsah

Flag	Význam
<code>--add-dir PATH</code>	Přidá další adresář (mimo working directory), který Claude smí číst. Lze opakovat.
<code>-w BRANCH, --worktree BRANCH</code>	Spustí relaci v izolovaném git worktree na uvedené větvi. Skvělé pro paralelní experimenty bez konfliktu.

```

claude --add-dir /home/tomas/shared-lib --add-dir /home/tomas/api-types
claude --worktree feature/oauth-refactor

```

3.5. Nástroje (allow/deny inline)

Flag	Význam
<code>--allowed-tools "Read,Write,Bash(npm run *)"</code>	Whitelist nástrojů na úrovni této relace.
<code>--disallowed-tools "WebFetch,Bash(rm:*)"</code>	Blacklist. Aplikuje se i v rámci povolení z <code>settings.json</code> .

Patterny jsou stejné jako v `settings.json` (viz kapitola 10).

3.6. Permission modes

Flag	Význam
--permission-mode MODE	default, acceptEdits, plan, bypassPermissions.
--permission-prompt-tool MCP_TOOL	Deleguje rozhodování o oprávněních na zadaný MCP tool.
--dangerously-skip-permissions	 Vypne kontroly úplně. Pouze v sandboxu (kontejner, VM).

```
# Bezpečný plan-only sběr informací
claude --permission-mode plan -p "Navrhni migrace z EF6 na EF Core"

# Plně automatizovaný režim v Docker kontejneru
docker run --rm -v $PWD:/app claude-image \
  claude -p "Aplikuj všechny TODO komentáře" \
  --dangerously-skip-permissions
```

3.7. Systémový prompt

Tyto čtyři flagy umožňují přepsat nebo doplnit systémový prompt, který určuje chování Claude Code.

Flag	Význam
--system-prompt TEXT	Plně nahradí systémový prompt textem. Vzájemně exklusivní s --system-prompt-file.
--system-prompt-file PATH	Plně nahradí systémový prompt obsahem souboru.
--append-system-prompt TEXT	Přidá text na konec stávajícího systémového promptu.
--append-system-prompt-file PATH	Přidá obsah souboru na konec systémového promptu.

 Plné nahrazení systémového promptu **vypne** velkou část toho, co dělá Claude Code „Claude Code“. Většinou chcete `--append-*`.

```
# Doplnění pravidel pro firemní review
claude --append-system-prompt-file ./claude/coding-standards.md
```

3.8. MCP a konfigurace

Flag	Význam
--mcp-config FILE_OR_JSON	Načte MCP servery z konkrétního souboru / JSON stringu.

--strict-mcp-config	Použije <i>pouze</i> MCP servery z --mcp-config, ostatní zdroje (settings) ignoruje.
--settings FILE (alias --config FILE)	Vynucený settings.json.

```
claude --mcp-config ./mcp-prod.json --strict-mcp-config
```

3.9. IDE a debug

Flag	Význam
--ide vscode/--ide jetbrains	Explicitně cílí integrované IDE; jinak autodetekce.
--debug	Vypíše system prompt, načítané CLAUDE.md, definice nástrojů, telemetrie.
--mcp-debug	Logovat raw JSON-RPC zprávy mezi Claudem a MCP servery.

3.10. Pomocné

Flag	Význam
--help	Stručná nápověda (⚠ není kompletní – některé flagy jsou nedokumentované).
--version	Verze CLI.

💡 Plnou referenci najdete v Příloze A.

4. Subcommands

Vedle hlavního `claude [PROMPT]` existují i samostatné podpříkazy, které se chovají jako jednorázové utility.

4.1. claude config

Otevře interaktivní tabulkový editor pro `settings.json`. Změny jsou aplikované okamžitě bez restartu, ESC vrátí nepotvrzené změny zpět.

```
claude config          # interaktivní UI
claude config get model # přečte konkrétní klíč
claude config set model claude-opus-4-6
```

4.2. claude mcp

Spravuje MCP servery. Detailně viz kapitolu 15.

```
claude mcp list
claude mcp add github node ~/.mcp/github/dist/index.js
claude mcp add stripe --transport sse https://mcp.stripe.com
claude mcp add-from-claude-desktop
claude mcp remove github
claude mcp get github
claude mcp reset-project-choices
```

4.3. claude doctor

Diagnostika instalace, MCP serverů, autentizace, oprávnění a aktualizací. V relaci dostupné jako `/doctor`. Pokud uvidíte červený status, stiskněte `f` pro auto-fix.

```
claude doctor
```

4.4. claude update

Stáhne a nainstaluje nejnovější verzi. Lze deaktivovat nastavením `DISABLE_AUTOUPDATER=true`.

4.5. claude migrate-installer

Migrate ze starší instalace (např. globální npm → managed installer).

4.6. claude login / claude logout

Přepnutí účtu Anthropic. V relaci dostupné jako `/login` a `/logout`.

4.7. claude install

Instalační asistent (typicky pro doinstalování pluginů či shell completions).

5. Vestavěné slash příkazy

Slash příkazy se spouští v interaktivní relaci. Pišete `/`, otevře se nabídka, dokončíte tabulátorem.

5.1. Správa relace

Příkaz	Funkce
<code>/help</code>	Zobrazí všechny dostupné příkazy včetně vlastních a MCP.
<code>/clear</code>	Vymaže celou konverzační historii.
<code>/compact</code>	Zkomprimuje historii do shrnutí (zachová klíčový kontext, uvolní tokeny).

/exit, /quit	Ukončí relaci.
/status	Aktuální session ID, model, účet, working dir, načtené plug-iny.
/cost	Aktuální cena relace v USD a počet tokenů.
/context	Podrobně rozepsaný kontext: kolik tokenů zabírá system prompt, paměť, nástroje, historie.

```
> /context
System prompt:      4 312 tokens
CLAUDE.md (project): 2 104 tokens
CLAUDE.md (user):   890 tokens
Tools:              3 011 tokens
Conversation:       14 770 tokens
Total:              25 087 / 200 000 (12,5 %)
```

5.2. Paměť

Příkaz	Funkce
/init	Vygeneruje CLAUDE.md ze struktury projektu (následně doporučeno upravit ručně).
/memory	Otevře CLAUDE.md (a CLAUDE.local.md) v \$EDITOR.
/consolidate-memory	(skill) Sloučí duplikáty, propíše zastaralé fakty, vyčistí index.

5.3. Model a konfigurace

Příkaz	Funkce
/model	Přepne model uprostřed relace (Haiku ↔ Sonnet ↔ Opus).
/config, /settings	Interaktivní editor settings.json.
/permissions	Přidá / odebere pattern z permissions.allow nebo deny.
/add-dir PATH	Přidá adresář k working directory za běhu.

5.4. Subagenti, skills, plugins, hooks, schedule

Příkaz	Funkce
/agents	Správa subagentů – tvorba, editace, mazání.
/skills	Správa skills (lokální + plug-in).
/plugins	Správa pluginů a marketplaces.
/hooks	Editor hooks v settings.json.

/schedule	(skill) Naplánuje úkol na opakované spuštění.
/output-style	Vybere output style (default, explanatory, learning, ...).

5.5. MCP

Příkaz	Funkce
/mcp	Status MCP serverů, OAuth flow, restart, debug.
/mcp__SERVER__PROMPT	Spuštění promptu vystaveného daným MCP serverem (např. /mcp__github__list_prs).

5.6. Vývojářské skill-driven příkazy

(Dostupné podle nainstalovaných skillů a pluginů.)

Příkaz	Funkce
/review	Code review aktuální větve / PR.
/security-review	Bezpečnostní revize neuložených změn.
/pr-comments	Stáhne komentáře z PR a vrátí je strukturovaně.
/init	Inicializace nového CLAUDE.md.
/debug	Asistent pro hledání chyb.

5.7. UI a integrace

Příkaz	Funkce
/ide	Připojí se k otevřenému VS Code / JetBrains.
/vim	Přepne Vim mode pro vstup.
/terminal-setup	Zapíše doporučené keybindings (Shift+Enter, Ctrl+J) do ~/.zshrc ~/.bashrc ~/.config/powershell/.
/keybindings	Editace ~/.claude/keybindings.json.
/theme	Výběr barevného schématu.
/export	Vyexportuje aktuální konverzaci do markdownu / JSON.

5.8. Maintenance

Příkaz	Funkce
/doctor	Diagnostika za běhu.
/update	Aktualizace Claude Code.

/login, /logout	Přepnutí účtu.
/bug	Otevře vstupní formulář pro nahlášení chyby (lze deaktivovat přes DISABLE_BUG_COMMAND).

6. Vlastní slash příkazy

Vlastní slash příkazy jsou markdown soubory s YAML frontmatterem. Claude Code je načte z těchto adresářů:

- **Project scope:** `.claude/commands/` (commitujte do git, sdílí se v týmu)
- **User scope:** `~/.claude/commands/` (osobní, napříč projekty)

6.1. Minimální command

```
---
description: Spustí lint a opraví možné formátovací chyby
allowed-tools: Bash(npm run lint*), Read, Edit
---

Najdi všechny lint chyby pomocí `npm run lint`, navrhní opravy a po schválení je proved.
```

Soubor `.claude/commands/lint-fix.md` se v relaci stane `/lint-fix`.

6.2. Frontmatter klíče

Klíč	Účel	Příklad
description	Krátký popis – zobrazuje se v /help a v navigaci.	Vygeneruj migraci EF Core
allowed-tools	Whitelist nástrojů (přepisuje session default).	Bash, Read, Edit
disallowed-tools	Blacklist nástrojů.	WebFetch
argument-hint	Hint do UI o očekávaných argumentech.	[ticket-id] [priority]
model	Vynucený model (haiku, sonnet, opus, inherit).	inherit

6.3. Argumenty: \$ARGUMENTS

V těle příkazu lze použít placeholder `$ARGUMENTS`. Vše, co uživatel napíše za jméno commandu, se do něj dosadí.

```
---
description: Připraví commit message podle Conventional Commits
argument-hint: [scope]
allowed-tools: Bash(git diff*), Bash(git log*)
---
```

Načti ``git diff --staged`` a vytvoř Conventional Commits zprávu se scopem ****\$ARGUMENTS****.

Vyhni se generickým slovesům jako "update" a "fix"; buď konkrétní.

Volání: `/commit-msg auth`

6.4. Inline shell a file references

Tělo příkazu se renderuje jako prompt. Lze do něj zapsat:

- `!{příkaz}` – Claude Code spustí shell příkaz a výsledek injektuje do promptu.
- `@cesta/k/souboru` – Claude přečte soubor a vloží jeho obsah.

```
---
description: Diagnostika selhání posledního CI běhu
allowed-tools: Bash(gh run*)
---
```

Toto je log posledního selhaného běhu:

!{gh run view --log-failed --limit 1}

A toto je relevantní workflow:

@.github/workflows/ci.yml

Vysvětli příčinu selhání a navrhní opravu.

6.5. Namespacing

Podadresáře se promítají do jména commandu:

- `.claude/commands/db/migrate.md` → `/db/migrate`
- `.claude/commands/security/audit.md` → `/security/audit`

6.6. Příklady reálných commandů

`/release-notes`

```
---
description: Vygeneruj release notes z commitů od posledního tagu
allowed-tools: Bash(git*), Read
---
```

*Stáhni ``git log $(git describe --tags --abbrev=0)..HEAD --pretty=format:"- %s" `` a převed`
do release notes podle Keep-a-Changelog formátu (Added / Changed / Fixed / Removed).*

/scaffold-test

```
---
description: Vygeneruj xUnit test pro daný soubor
argument-hint: [relativní/cesta/k/Souboru.cs]
allowed-tools: Read, Write, Glob
model: claude-sonnet-4-6
---
```

Načti soubor ``$ARGUMENTS``, identifikuj veřejné metody a třídy, a v projektu Tests/
vytvoř odpovídající xUnit test class. Použij FluentAssertions a Moq.

7. Konfigurační hierarchie a **settings.json**

7.1. Lokace souborů

Scope	Cesta	Commitovat?
Managed (Enterprise)	/Library/Application Support/ClaudeCode/managed-settings.json (macOS), C:\ProgramData\ClaudeCode\managed-settings.json (Windows), /etc/claude-code/managed-settings.json (Linux)	ne (řízené IT)
User	~/.claude/settings.json	ne
Project	.claude/settings.json	ano
Local	.claude/settings.local.json	ne (gitignore)

7.2. Kompletní kostra

```
{
  "$schema": "https://platform.claude.com/schemas/claude-settings.json",
  "model": "claude-opus-4-6",
  "fallbackModel": "claude-sonnet-4-6",
  "defaultHaikuModel": "claude-haiku-4-5-20251001",
  "cleanupPeriodDays": 14,
```

```

"includeCoAuthoredBy": false,
"autoMemoryEnabled": true,
"apiKeyHelper": "~/bin/get-anthropic-key.sh",

"permissions": {
  "defaultMode": "default",
  "additionalDirectories": ["/home/tomas/shared-lib"],
  "allow": [
    "Read",
    "Glob",
    "Grep",
    "Edit",
    "Write",
    "Bash(npm run *)",
    "Bash(dotnet *)",
    "Bash(git *)",
    "WebSearch",
    "WebFetch(domain:learn.microsoft.com)",
    "WebFetch(domain:docs.claude.com)"
  ],
  "deny": [
    "Read(./env)",
    "Read(./secrets/**)",
    "Read(./**/appsettings.Production.json)",
    "Bash(rm -rf*)",
    "Bash(curl*|*sh)"
  ],
  "ask": [
    "Bash"
  ]
},

"env": {
  "DOTNET_NOLOGO": "1",
  "ASPNETCORE_ENVIRONMENT": "Development"
},

"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash",
      "hooks": [
        { "type": "command", "command": ".claude/hooks/audit-bash.sh",

```

```

"timeout": 3000 }
    ]
  },
  ],
  "PostToolUse": [
    {
      "matcher": "Write|Edit",
      "hooks": [
        { "type": "command", "command": "dotnet format --include
\\$CLAUDE_TOOL_INPUT_FILE_PATH\\" }
      ]
    }
  ]
},

"mcpServers": {
  "github": {
    "type": "stdio",
    "command": "node",
    "args": [ "~/.mcp/github/dist/index.js" ],
    "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
  }
}
}

```

7.3. Klíče top-level

- **model** : výchozí model relace.
- **fallbackModel** : záloha při výpadku/rate-limitu.
- **defaultHaikuModel** / **defaultSonnetModel** / **defaultOpusModel** : explicitní mapování rodiny modelů na konkrétní revize.
- **cleanupPeriodDays** : po kolika dnech se mažou staré transkripty.
- **includeCoAuthoredBy** : přidává **Co-Authored-By: Claude** **<noreply@anthropic.com>** do **git commit** . Doporučeno **false** v týmech, kde recenzent chce vidět autora kódu.
- **autoMemoryEnabled** : Claude si může psát vlastní memo do **~/.claude/projects/<proj>/memory/** . Užitečné, ale stojí kontext.
- **apiKeyHelper** : cesta na skript, který vrátí API klíč (např. z firemního vaultu). Zavolá se při startu relace.

7.4. permissions

Detailní rozbor je v kapitole 10 (Permissions). Tady jen přehled:

- `defaultMode` : výchozí permission mode (`default` / `acceptEdits` / `plan` / `bypassPermissions`).
- `allow` / `deny` / `ask` : pole regex-like patternů.
- `additionalDirectories` : rozšíření o další adresáře.

7.5. env

Mapa proměnných injektovaných do session a do shellů spouštěných přes Bash tool. Hodnoty mohou být literály, ale **existující proměnné v shellu mají přednost**.

```
"env": {
  "ASPNETCORE_ENVIRONMENT": "Development",
  "PATH": "${PATH}:/opt/dotnet/sdk"
}
```

7.6. hooks

Registr lifecycle hooks. Detailně v kapitole 14.

7.7. mcpServers

Konfigurace MCP serverů (alternativa k `claude mcp add`). Detailně v kapitole 15.

7.8. JSON schema podpora v IDE

Přidejte do souboru:

```
"$schema": "https://platform.claude.com/schemas/claude-settings.json"
```

VS Code i JetBrains potom poskytnou autocompletion a validaci.

8. Paměť: CLAUDE.md a importy

8.1. Hierarchie

Soubor	Načítá se	Účel
~/.claude/CLAUDE.md	vždy	Globální preference (např. „odpovídej česky, kód komentuj anglicky“).
<projekt>/CLAUDE.md	v daném projektu	Architektura, konvence, lokální buildy.
<projekt>/CLAUDE.local.md	v daném projektu, gitignored	Osobní dodatky.

<podadresář>/CLAUDE.md	když Claude pracuje v podstromu	Specifika subprojektu.
------------------------	---------------------------------	------------------------

Všechny dohromady se konkatenují do system promptu při startu (resp. při změně working dir).

8.2. Co psát do **CLAUDE.md**

Doporučená struktura:

Cli Reference (interní knihovna)

Knihovna pro generování CLI fasád v .NET 9. Cílová publika: backend teamy, kteří chtějí vystavit operations endpoints přes terminal.

Tech stack

- .NET 9, C# 13
- xUnit + FluentAssertions
- Spectre.Console pro UI vrstvu
- GitHub Actions pro CI

Konvence

- Žádné `var` v public API (preferujeme explicitní typy).
- Async metody mají suffix `Async` a vždy přijímají `CancellationToken`.
- Nullable reference types jsou zapnuté ve všech projektech.
- DI registrace v `ServiceCollectionExtensions`, jméno metody `Add{Feature}`.

Build a test

```
dotnet restore dotnet build --no-restore dotnet test --collect:"XPlat Code Coverage"
```

Architektonická pravidla

- Žádný kód nezávisí na konkrétním ORM – data access vrstva je za rozhraním.
- Konfigurace přes `IOptions<T>`, nikdy přímo `IConfiguration` v service vrstvě.
- Logování přes `ILogger<T>` se source-generated metodami (`LoggerMessage`).

Co nedělat

- Neměnit veřejné API bez bumpnutí major verze.
- Neukládat tajemství do `appsettings*.json` – patří do uživatelských secrets / vaultu.

8.3. Importy: @cesta/k/souboru

Do `CLAUDE.md` lze inline vložit obsah jiného souboru:

```
## Coding standards
```

```
@docs/coding-standards.md
```

```
## Glosář pojmů
```

```
@docs/glossary.md
```

Importy se rozbalí při startu, čímž lze sdílet pravidla mezi více projekty.

8.4. Hygiena CLAUDE.md

- **Stručně.** Každá věta žere kontext.
- **Aktualizujte.** `/init` umí zregenerovat základ; ručně dolad'ujte.
- **Nepište do CLAUDE.md tajemství** – soubor jde do gitu.
- **Lokální preference** patří do `CLAUDE.local.md`.

9. Environment proměnné

Proměnné lze nastavit v shellu, v `settings.json` (`env` klíč), nebo ve scopovaných `.env` souborech načítaných hookem.

9.1. Autentizace a API endpoint

Proměnná	Význam
ANTHROPIC_API_KEY	API klíč pro přímé volání Anthropic API.
ANTHROPIC_AUTH_TOKEN	Bearer token (alternativa k ANTHROPIC_API_KEY, pro proxy).
ANTHROPIC_BASE_URL	Vlastní endpoint (default <code>https://api.anthropic.com</code>).
ANTHROPIC_CUSTOM_HEADERS	JSON s vlastními HTTP hlavičkami.

9.2. Modely

Proměnná	Význam
ANTHROPIC_MODEL	Globální výchozí model.
ANTHROPIC_DEFAULT_HAIKU_MODEL	Konkrétní revize Haiku.
ANTHROPIC_DEFAULT_SONNET_MODEL	Konkrétní revize Sonnet.

ANTHROPIC_DEFAULT_OPUS_MODEL	Konkrétní revize Opus.
------------------------------	------------------------

9.3. Cloud providery

Proměnná	Význam
CLAUDE_CODE_USE_BEDROCK	true = volat přes AWS Bedrock.
CLAUDE_CODE_USE_VERTEX	true = volat přes Google Vertex AI.
CLAUDE_CODE_SKIP_BEDROCK_AUTH	Přeskočí AWS auth (pokud máte vlastní mechanismus).
AWS_REGION	AWS region pro Bedrock.
CLOUD_ML_REGION	Google region pro Vertex AI.
ANTHROPIC_VERTEX_PROJECT_ID	Projekt pro Vertex AI.

9.4. Limity nástrojů

Proměnná	Význam
BASH_DEFAULT_TIMEOUT_MS	Výchozí timeout pro Bash tool (ms).
BASH_MAX_TIMEOUT_MS	Maximální timeout pro Bash.
BASH_MAX_OUTPUT_LENGTH	Maximální délka výstupu (znaky), default ~500 KB.
CLAUDE_CODE_MAX_OUTPUT_TOKENS	Limit na output tokeny.
MCP_TIMEOUT	Timeout startu MCP serveru.
MCP_TOOL_TIMEOUT	Timeout pro volání jednotlivého MCP toolu.
MAX_THINKING_TOKENS	Limit pro extended thinking.

9.5. Behaviorální flagy

Proměnná	Význam
DISABLE_AUTOUPDATER	Vypne automatickou aktualizaci.
DISABLE_TELEMETRY, CLAUDE_CODE_DISABLE_TELEMETRY	Vypne telemetrii.
DISABLE_BUG_COMMAND	Skryje /bug v menu.
DISABLE_COST_WARNINGS	Skryje varování o nákladech.
DISABLE_ERROR_REPORTING	Vypne odesílání pádových reportů.
DISABLE_NON_ESSENTIAL_MODEL_CALLS	Vypne pomocná volání modelu (titulky, summaries).
DISABLE_PROMPT_CACHING	Vypne prompt caching (může být beta).

9.6. Bezpečnostní doporučení

🛡 Nikdy nepřidávejte `ANTHROPIC_API_KEY` do `settings.json` – soubor jde do gitu. Použijte:

- shell `export` v `~/.zshrc` / `$PROFILE` ,
- `apiKeyHelper` (skript, který vyzvedne klíč z vaultu),
- správce tajemství (1Password CLI, Bitwarden CLI, AWS Secrets Manager).

10. Permissions a režimy oprávnění

10.1. Permission modes

Claude Code má pět režimů oprávnění:

Režim	Chování	Kdy použít
default	Před každým „write/exec“ akcí se ptá.	Sensitivní kód, neznámé projekty.
acceptEdits	Auto-schválí editace souborů a běžné bash (<code>mkdir</code> , <code>cp</code> , <code>mv</code> , ...).	Polo-automatický rytmus, mainline development.
plan	Read-only. Claude smí číst, hledat, plánovat, ale needitovat ani spouštět příkazy.	Architektura, audit, brainstorm.
bypassPermissions	Vypne všechny kontroly.	🛡 Pouze v sandboxu (kontejner, VM).

⚠ Režim přepínáte buď flagem `--permission-mode` , hodnotou v `settings.json` (`permissions.defaultMode`), nebo za běhu přes Shift+Tab (cyklicky).

10.2. permissions v settings.json

```
"permissions": {
  "defaultMode": "default",
  "additionalDirectories": ["/home/tomas/shared-lib"],
  "allow": [
    "Read",
    "Glob",
    "Grep",
    "Bash(npm run *)",
    "Bash(git diff*)",
    "WebFetch(domain:docs.claude.com)"
  ],
}
```

```

"deny": [
  "Read(/secrets/**)",
  "Bash(rm -rf*)",
  "Bash(curl*|*sh)"
],
"ask": [
  "Bash",
  "Write"
]
}

```

10.3. Pattern syntax

Tools se omezují stringem ve formátu `Tool(pattern)` :

- `Bash(npm run *)` – povolené jakékoli `npm run ...`.
- `Bash(git commit:*)` – obecný `git commit` se vším potomstvem.
- `Read(/secrets/**)` – read na celý strom `/secrets/`.
- `WebFetch(domain:github.com)` – fetch jen na github.com.
- `mcp__github` – povolené volání jakéhokoli toolu z `github` MCP serveru.

První matchující pravidlo vyhraje, `deny` má přednost před `allow`.

10.4. Custom permission prompt přes MCP

Pro firemní/audit prostředí lze rozhodování delegovat na vlastní MCP tool:

```

claude --permission-prompt-tool mcp__corp-policy__decide

```

MCP server `corp-policy` musí vystavit tool `decide`, který přijme JSON s popisem akce a vrátí `allow` / `deny` s důvodem.

10.5. Trust dialog pro nové projekty

Když Claude poprvé vstoupí do nového adresáře, otevře se „trust dialog“. Zvolením *Trust* projekt Claude vidí vše, co je nakonfigurováno v `.claude/`. Bez trustu se neaktivují project-scope MCP servery, agenti a hooks.

10.6. Bezpečné automatizace

 **Nikdy nespouštějte `--dangerously-skip-permissions` na hostu**, kde máte přístup k SSH klíčům, vaultům, produkčním credentials.

Bezpečné varianty:

- Devcontainer / Codespaces: Claude běží v ephemerním kontejneru bez sítě.
- Vlastní Docker image s read-only mountem repa.

- WSL2 instance dedikovaná pro Claude.

11. Vestavěné nástroje (Tools)

Claude Code má integrovanou sadu nástrojů, které volá podle potřeby. Tato kapitola je referencí; reálné použití koordinuje sám Claude.

11.1. File operations

Tool	Účel	Pozn.
Read	Načte soubor (text, image, PDF, ipynb). Pracuje s absolutními cestami.	Nečte adresáře.
Write	Vytvoří soubor, případně přepíše.	Vyžaduje předchozí Read, pokud soubor existuje.
Edit	Replace dvojice (old → new) v souboru.	replace_all: true pro hromadnou změnu.
MultiEdit	(legacy) Více editů v jednom volání.	V novějších verzích nahrazeno seriovým Edit.

11.2. Vyhledávání

Tool	Účel
Glob	Pattern matching cest (**/*.cs).
Grep	ripgrep (regex, multiline, glob filter).

11.3. Notebooks

Tool	Účel
NotebookRead	Čtení Jupyter .ipynb (cell-aware).
NotebookEdit	Úprava buněk Jupyteru.

11.4. Web

Tool	Účel
WebFetch	Stažení stránky a vrácení markdownu.
WebSearch	Vyhledávání v internetu.

11.5. Shell

Tool	Účel
------	------

Bash	Spouští shell příkazy v perzistentní session.
BashOutput	Čte stdout / stderr běžícího Bash jobu.
KillBash	Ukončí bash job.

11.6. Plánování a delegace

Tool	Účel
TodoWrite	Zápis aktuálního TODO listu (sledování progresu).
Task	Spuštění subagenta.
ExitPlanMode	Programatický exit z plan modu po schválení plánu.
SlashCommand	Vyvolání slash příkazu z agenta.

11.7. Vlastní nástroje přes MCP

Vše ostatní (Slack, Linear, GitHub, Jira, Postgres, Stripe, ...) přibývá přes MCP servery. Volá se jako `mcp__server__tool` (kapitola 15).

12. Subagenti

Subagent je samostatný „mini-Claude“ s vlastním system promptem, sadou nástrojů, modelem a kontextem. Hlavní agent ho deleguje k focusovanému úkolu a vrátí se mu jen výsledek – mezivolání nezatěžují hlavní context.

12.1. Lokace

Scope	Cesta
Project	<code>.claude/agents/<name>.md</code>
User	<code>~/.claude/agents/<name>.md</code>
Managed	<code>enterprise-managed</code>

12.2. Frontmatter

```
---
name: csharp-test-writer
description: Píše xUnit testy pro C# kód (FluentAssertions + Moq). Volej, když uživatel zmíní "testy", "xunit", nebo když přidává novou veřejnou metodu.
tools:
  - Read
  - Glob
```

- Grep
- Edit
- Write
- Bash(dotnet test*)

disallowedTools:

- WebFetch
- WebSearch

model: claude-sonnet-4-6

permissionMode: acceptEdits

maxTurns: 30

color: cyan

Jsi expert na unit testing v C# / .NET 9.

Pravidla

- Používej xUnit, FluentAssertions a Moq.
- Nikdy nepiš mocky pro statické metody – preferuj refaktor na rozhraní.
- Testovací třídy umisťuj do `Tests/<Projekt>.Tests/<Stejná struktura>` jako produkční kód.
- Naming: `MethodName\_Scenario\_ExpectedBehavior`.
- Vždy přidej `[Fact]` nebo `[Theory]`.

Postup

1. Načti zadaný produkční soubor.
2. Identifikuj veřejné metody a jejich kontrakty.
3. Pro každou metodu navrhni test cases (happy path, null arg, hraniční hodnota, exception).
4. Napiš testovací soubor.
5. Spusť `dotnet test --filter` a potvrď zelený run.

12.3. Klíče frontmatteru

Klíč	Význam
name	Identifikátor (kebab-case).
description	Co umí + kdy se má volat (autodiscovery).
tools / allowed-tools	Whitelist nástrojů.
disallowedTools	Blacklist.
model	haiku / sonnet / opus / inherit / konkrétní revize.

permissionMode	viz kapitola 10.
maxTurns	Strop.
skills	Naloadované skills v rámci agenta.
memory	Cesta na vlastní memory soubor.
color	Barva v terminálu.
effort	Hint o náročnosti (low/medium/high).
isolation	worktree pro práci v oddělené větvi.

12.4. Volání agenta

Explicitně:

„Use the `csharp-test-writer` agent to add tests for `OrderService` .”

Implicitně: hlavní agent se rozhodne sám podle `description` . Proto piště description jasně a uvádějte trigger (,,volej, když...”).

12.5. Životní cyklus

1. Hlavní agent zavolá `Task` tool s parametry (subagent name, prompt).
2. Spustí se nová izolovaná kontextová relace s system promptem agenta.
3. Subagent pracuje, používá své tooly, případně hlásí progres.
4. Vráť finální zprávu (hlavní agent vidí jen tu).
5. Hlavní agent pokračuje v práci.

12.6. Worktree izolace

Pro paralelní úkoly bez konfliktu:

```
---
name: parallel-fixer
isolation: worktree
...
---
```

Spuštěním tohoto agenta vznikne nový git worktree (větev) a všechny změny zůstanou izolované od hlavní pracovní kopie.

12.7. `/agents`

Slash příkaz `/agents` otevře interaktivní správce: vytvořit, editovat, smazat, spustit.

13. Skills

Skills jsou „instructional packages“: markdown s YAML frontmatterem (a volitelně podadresáři `references/` , `scripts/`). Claude o nich průběžně ví a sám se rozhoduje, kdy je použít. Klíčový mechanismus se nazývá *progressive disclosure*.

13.1. Lokace

Scope	Cesta
User	~/.claude/skills/<skill-name>/SKILL.md
Project	.claude/skills/<skill-name>/SKILL.md
Plugin	uvnitř instalovaného pluginu

13.2. Struktura skillu

```
my-skill/
  SKILL.md                # frontmatter + instrukce
  references/              # volitelně, načítá se on-demand
    api-shapes.md
    style-guide.md
  scripts/                 # volitelně
    setup.sh
    prepare-data.py
```

13.3. SKILL.md frontmatter

```
---
name: ef-core-migrations
description: Generuje a kontroluje EF Core migrace v .NET projektech.
Spouštěj při zmínkách "migrace", "DbContext", "Add-Migration" nebo když se
mění modelové třídy v Entities/.
required:
  - Read
  - Edit
  - Bash(dotnet ef *)
optional:
  - WebSearch
model: inherit
---
```

Jsi expert na EF Core 9 migrace v .NET.

Postup

1. Identifikuj ``DbContext`` v projektu.
2. Zjisti, zda jsou pending změny (``dotnet ef migrations list``).
3. Navrhni jméno migrace v *PascalCase* popisující záměr.
4. Spust' ``dotnet ef migrations add <Name>`` a zkontroluj vygenerovanou *Up/Down*.
5. Pokud *Down* obsahuje destruktivní operace bez kontroly, navrhni mitigaci.
6. Před *commitem* zavolej ``dotnet build`` a ``dotnet ef database update`` v *dev DB*.

Kdy NE

- Pokud projekt používá *Dapper* bez *EF*, *NE*spouštěj.
- Pokud je v *repu* adresář ``Migrations.Manual/``, ptej se uživatele.

Reference

Pro detail standardních polí čti `@references/style-guide.md`.

13.4. Progressive disclosure

V system promptu zabírá skill jen *frontmatter* (jméno + description). Tělo **SKILL.md** se načte, až když Claude vyhodnotí, že je relevantní. References se načítají dále on-demand.

⇒ V systému s 30 skillsy nepřijdete o ~150 K tokenů – Claude o nich „ví“, ale text drží mimo kontext.

13.5. Jak Claude vybírá skill

Rozhoduje na základě:

1. Jaké slovo / nástroj uživatel použil (key terms v **description**).
2. Jaké soubory upravuje.
3. Předchozí kontext (často zmíněné fráze).

Proto v **description** uvádějte **přesné trigger**y (slova, cesty, názvy nástrojů).

13.6. Skill-creator

Plugin obsahující skill **skill-creator** , který umí:

- vygenerovat kostru nového skillu,
- spustit eval (skóre, jak dobře se aktivuje na testovacích promptech),
- benchmarkovat výkon.

13.7. Příklady užitečných skillů (reálné)

- **docx , pdf , pptx , xlsx** – tvorba a úprava office dokumentů.
- **ef-core-migrations** – migrace .NET projektů.

- `consolidate-memory` – pravidelná hygiena `CLAUDE.md` .
- `theme-factory` – branding artefaktů (slides, reporty).
- `mcp-builder` – průvodce stavbou MCP serveru.

14. Hooks

Hooks jsou shell skripty napojené na životní cyklus relace. Slouží k auditování, vynucování pravidel, automatickému formátování a integracím (např. notifikace do Slacku).

14.1. Typy hooks

Hook	Kdy se spouští
PreToolUse	Před voláním kteréhokoli toolu. Lze blokovat / modifikovat vstup.
PostToolUse	Po úspěšném volání toolu.
UserPromptSubmit	Když uživatel odešle prompt.
Notification	Před zobrazením OS notifikace.
Stop	Když Claude dokončí odpověď.
SubagentStop	Když dokončí subagent.
PreCompact	Před <code>/compact</code> .
SessionStart	Na začátku relace.
SessionEnd	Při ukončení relace.

14.2. Konfigurace v `settings.json`

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash",
      "hooks": [
        { "type": "command", "command": ".claude/hooks/audit-bash.sh",
"timeout": 3000 }
      ]
    },
    {
      "matcher": "Write|Edit",
      "hooks": [
        { "type": "command", "command": ".claude/hooks/no-secrets.sh" }
      ]
    }
  ]
}
```

```

    }
  ],
  "PostToolUse": [
    {
      "matcher": "Write|Edit",
      "hooks": [
        { "type": "command", "command": "dotnet format --include
\\$CLAUDE_TOOL_INPUT_FILE_PATH\\" }
      ]
    }
  ],
  "UserPromptSubmit": [
    {
      "hooks": [
        { "type": "command", "command": ".claude/hooks/log-prompt.sh" }
      ]
    }
  ]
}

```

14.3. Vstup / výstup

Hook dostane na stdin JSON s eventem (tool name, parametry, soubor, stdout). Vrací JSON na stdout:

```

{
  "decision": "allow",
  "reason": "audit logged"
}

```

Hodnoty **decision** :

Hodnota	Účinek
allow	Pokračuj.
block	Zakaž akci, vrať reason Claudovi.
modify	Pokračuj s upraveným vstupem v modifiedInput.

Exit code $\neq 0$ znamená *block* + log error.

14.4. Užitečné environment proměnné v hooku

Proměnná	Význam
----------	--------

CLAUDE_TOOL_NAME	Jméno tool (Read, Bash, ...).
CLAUDE_TOOL_INPUT_FILE_PATH	Cesta editovaného souboru.
CLAUDE_TOOL_INPUT_COMMAND	Bash příkaz (u Bash).
CLAUDE_PROMPT_TEXT	Text uživatelského promptu.
CLAUDE_SESSION_ID	UUID relace.

14.5. Příklady

audit-bash.sh – audit log:

```
#!/usr/bin/env bash
read -r INPUT
echo "$(date -Iseconds) $CLAUDE_SESSION_ID $INPUT" >> ~/.claude/audit-
bash.log
echo '{"decision":"allow"}'
```

no-secrets.sh – blok proti secret commitům:

```
#!/usr/bin/env bash
read -r INPUT
FILE=$(echo "$INPUT" | jq -r '.input.file_path')
if grep -E '(AKIA[0-9A-Z]{16}|sk-[a-zA-Z0-9]{32,})' "$FILE" > /dev/null 2>&1;
then
    echo '{"decision":"block","reason":"Detected secret in file."}'
else
    echo '{"decision":"allow"}'
fi
```

prettier-on-write.sh – auto formát:

```
#!/usr/bin/env bash
read -r INPUT
FILE=$(echo "$INPUT" | jq -r '.input.file_path')
case "$FILE" in
    *.cs) dotnet format --include "$FILE" >/dev/null 2>&1 ;;
    *.ts|*.tsx|*.js|*.jsx) npx prettier --write "$FILE" >/dev/null 2>&1 ;;
esac
echo '{"decision":"allow"}'
```

14.6. Dobré zvyky pro hooks

-  **Idempotence.** Hook se může spustit opakovaně.
-  **Krátké timeouty.** Default 60 s je hodně; pro `PreToolUse` nastavujte 1–3 s.
-  **Logujte do souboru.** Stdout slouží pro decision JSON, ne pro debug výpisy.

15. MCP (Model Context Protocol)

MCP je otevřený protokol, kterým Claude Code komunikuje s externími servery (nástroji, datovými zdroji, prompty).

15.1. Tři typy serverů

Typ	Komunikace	Příklad
stdio	spuštěný proces, stdin/stdout JSON-RPC	lokální nástroj v Pythonu / Node
sse	Server-Sent Events přes HTTPS	hostovaný cloud service
http	klasické HTTP	jednoduché API wrappery

15.2. Přidávání serverů

```
# stdio
claude mcp add github node ~/.mcp/github/dist/index.js

# sse
claude mcp add stripe --transport sse https://mcp.stripe.com/api

# http
claude mcp add openai --transport http https://api.example.com/mcp

# z JSON
claude mcp add-json myserver '{
  "type": "stdio",
  "command": "uvx",
  "args": ["my-mcp-server", "--config", "/etc/myserver.yaml"]
}'

# import z Claude Desktop
claude mcp add-from-claude-desktop
```

15.3. Scope

```

claude mcp add --scope local github ... # jen tahle relace v projektu
claude mcp add --scope project github ... # commitnuto v .mcp.json
claude mcp add --scope user github ... # globálně přes settings.json

```

15.4. .mcp.json

```

{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "node",
      "args": ["/opt/mcp/github/dist/index.js"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "linear": {
      "type": "sse",
      "url": "https://mcp.linear.app/sse",
      "headers": { "Authorization": "Bearer ${LINEAR_API_KEY}" }
    }
  }
}

```

⚠ Project-scope servery vyžadují **trust** projektu (kapitola 10.5).

15.5. Příkazy v relaci

```

/mcp # status, OAuth flow, restart
/mcp__github__list_prs # spuštění promptu vystaveného serverem 'github'
@mcp__github__readme # vložení resource do promptu

```

15.6. Autentizace

Mechanismus	Použití
Env proměnné	env blok v konfiguraci.
OAuth	/mcp zahájí flow, výsledný token je uložen v keystore.
Bearer header	headers v .mcp.json (alt.: --mcp-config s tajemstvím v souboru mimo git).

🛡 Tokeny ukládejte do user-scope **settings.json** nebo do správce hesel; ne do project **.mcp.json**.

15.7. Debug

```
claude --mcp-debug # raw JSON-RPC trasy
```

V relaci `/mcp` zobrazí stav, případné chyby a umožní restart serverů.

15.8. Vlastní MCP server

Pro stavbu vlastního serveru použijte skill `mcp-builder` nebo SDK:

- TypeScript: `@modelcontextprotocol/sdk`
- Python: `mcp` (FastMCP třída)

Skeleton (TypeScript):

```
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";

const server = new Server({ name: "my-tool", version: "0.1.0" });

server.setRequestHandler("tools/list", async () => ({
  tools: [{ name: "echo", description: "Vrátí vstup." , inputSchema: { type:
"object" } }]
}));
server.setRequestHandler("tools/call", async (req) => {
  if (req.params.name === "echo") {
    return { content: [{ type: "text", text:
JSON.stringify(req.params.arguments) }] };
  }
});

await server.connect(new StdioServerTransport());
```

16. Plugins a marketplaces

Plugin je distribuovatelný balíček obsahující kombinaci skills, agentů, slash commandů, hooks a MCP konfigurací.

16.1. Lokace

Scope	Cesta
User	~/.claude/plugins/<plugin>/

Project	.claude/plugins/<plugin>/
---------	---------------------------

16.2. Marketplaces

Marketplace je registr pluginů. Anthropic provozuje oficiální, ale lze provozovat i vlastní (vnitrofiremní git repository s indexem).

16.3. /plugins v relaci

- Vyhledávání instalovaných pluginů.
- Instalace nových (z URL, marketplace).
- Zapnutí / vypnutí pluginu.
- Reload po editaci.

16.4. Struktura pluginu

```
my-plugin/  
  plugin.json          # manifest  
  skills/  
  agents/  
  commands/  
  hooks/  
  mcp/  
  README.md
```

plugin.json :

```
{  
  "name": "my-plugin",  
  "version": "0.2.0",  
  "description": "Sada pomocníků pro .NET týmy.",  
  "author": "Acme Corp",  
  "tags": ["dotnet", "csharp", "ef-core"]  
}
```

16.5. Vyhledávání skillů a pluginů

Z relace lze přes `/skills` a `/plugins` rychle zjistit, co je k dispozici. Pro nesplnitelné dotazy lze sáhnout do registru: existují MCP tooly `search_plugins`, `suggest_plugin_install`.

17. IDE integrace

17.1. VS Code

- Oficiální rozšíření z Marketplace (Anthropic Claude Code).

- Otevírá panel s chatem (volitelně side / bottom).
- Mention souborů přes `@`, slash příkazy přes `/`.
- Diff viewer s tlačítka *Accept* / *Reject* pro každou změnu.
- Diagnostiky (lint warnings, errors) jsou posílány Claudovi automaticky.
- `Ctrl+Esc` (macOS `Cmd+Esc`) – přepnutí mezi editorem a panelem.

17.2. JetBrains (IntelliJ, Rider, PyCharm...)

- Plugin orchestruje CLI běžící v IDE terminálu.
- IDE diff viewer pro review.
- Selection context: vybraný kód se přiloží.
- Debugger / problems integrace (částečně, závisí na IDE).

17.3. `/ide` v externím terminálu

Pokud běžíte `claude` v iTerm2 / Windows Terminal a máte otevřené VS Code, spuštěním `/ide` se Claude napojí na editor (číst aktivní soubor, otvírat diff).

17.4. Nastavení preferovaného IDE

```
claude --ide vscode
# nebo:
claude --ide jetbrains
```

18. Headless režim, CI/CD a automatizace

18.1. Základ

Headless = `claude -p "PROMPT"` :

- Není REPL, jen jeden požadavek.
- Vystupuje na stdout (text/JSON/stream-json).
- Návrátový kód: 0 = úspěch, $\neq 0$ = chyba.

18.2. GitHub Actions

```
name: PR Review by Claude
on:
  pull_request:
    types: [opened, synchronize, reopened]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
```

```

  contents: read
  pull-requests: write
steps:
  - uses: actions/checkout@v4
    with:
      fetch-depth: 0

  - name: Install Claude Code
    run: npm install -g @anthropic-ai/claude-code

  - name: Run review
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
    run: |
      claude -p "Proveď code review této PR. Vrať seznam problémů ve
formátu markdown." \
        --output-format json \
        --permission-mode plan \
        --max-turns 12 \
        --add-dir . \
        > review.json

  - name: Post comment
    uses: marocchino/sticky-pull-request-comment@v2
    with:
      path: review.json

```

18.3. Bezpečnost v CI

- Klíče vždy přes secret store (GH Secrets, AWS Secrets Manager).
- `--permission-mode plan` pro read-only review.
- `--strict-mcp-config ./mcp-ci.json` (jen MCP servery vhodné pro CI).
- Ephemeral runner / kontejner = lze použít `--dangerously-skip-permissions` pouze tady.

18.4. Plánované (opakované) spouštění

Claude Code lze pouštět periodicky třemi způsoby. Doporučení v praxi: pro úkoly vázané na uživatelskou Cowork relaci použijte **built-in skill /schedule**, pro serverový provoz **systemd timer** (lepší logování, retry, izolace), pro rychlé "quick-and-dirty" prototypy **cron**.

18.4.1. Built-in skill /schedule

Skill **schedule** (součást **anthropic-skills** plug-inu, dostupný v Cowork i v Claude Code) registruje opakované úkoly v rámci uživatelského účtu – běží na infrastruktuře Anthropic, ne lokálně. Vhodné pro úkoly, které pracují s MCP konektory připojenými k uživateli (Slack, GitHub, Linear, Drive).

Spuštění v REPL:

```
/schedule
```

Skill se zeptá na:

1. **Cron expression** nebo human-friendly interval (`every Monday 09:00 Europe/Prague`).
2. **Prompt** (co má agent dělat).
3. **MCP konektory**, které má úloha vidět.
4. **Notifikační kanál** (email / Slack DM).

Vhodné případy:

- Pondělní digest commitů z GitHubu poslaný do Slacku.
- Denní kontrola open dependabot PR.
- Hodinový sweep TODO/FIXME komentářů s nahlášením do Linearu.
- Týdenní rollup metrik z Datadogu do Notion stránky.

⚠ Skill běží v Cowork sandboxu, **nemá** přístup k lokálnímu disku ani k repu na vašem stroji. Pro úkoly, které musí pracovat s lokálním pracovním adresářem (`claude -p` nad rozpracovaným repem), použijte cron / systemd timer níže.

18.4.2. Linux cron

Klasický crontab + headless `claude -p` . Minimální verze:

```
# crontab -e
0 7 * * 1 cd /opt/projects/api && /usr/local/bin/claude -p "Vygeneruj týdenní
digest commitů od minulého pondělí ve formátu markdown a ulož ho do
/var/log/claude/digest-$(date +%F).md" --permission-mode plan --max-turns 8
--output-format text >> /var/log/claude/cron.log 2>&1
```

V praxi je takový one-liner křehký. Doporučený vzor – wrapper skript, který crontab jen volá:

```
#!/usr/bin/env bash
# /opt/claude-jobs/digest.sh
set -euo pipefail

# --- Konfigurace ---
PROJECT_DIR="/opt/projects/api"
LOG_DIR="/var/log/claude"
LOCK_FILE="/var/lock/claude-digest.lock"
JOB_ID="digest-$(date +%F-%H%M)"

# --- Secrets (z roota souboru, ne z env) ---
export ANTHROPIC_API_KEY="$(cat /etc/claude/api-key)"
```

```

chmod 600 /etc/claude/api-key # idempotentní jistota

# --- Idempotence: zabránění souběhu ---
exec 9>"$LOCK_FILE"
if ! flock -n 9; then
    echo "[$JOB_ID] Předchozí běh ještě běží, končím." >&2
    exit 0
fi

# --- Vlastní běh ---
mkdir -p "$LOG_DIR"
cd "$PROJECT_DIR"

# Retry s exponential backoff (3 pokusy)
for attempt in 1 2 3; do
    if claude -p "$(cat /opt/claude-jobs/prompts/digest.txt)" \
        --permission-mode plan \
        --max-turns 12 \
        --output-format json \
        --strict-mcp-config /etc/claude/mcp-prod.json \
        > "$LOG_DIR/$JOB_ID.json" \
        2> "$LOG_DIR/$JOB_ID.err"
    then
        echo "[$JOB_ID] OK (pokus $attempt)"
        exit 0
    fi
    sleep $((attempt * 30))
done

echo "[$JOB_ID] Selhalo po 3 pokusech" >&2
exit 1

```

Crontab pak jen:

```
0 7 * * 1 /opt/claude-jobs/digest.sh
```

💡 Důležité detaily:

- **PATH v cronu** je minimální. Vždy `/usr/local/bin/claude` plnou cestou nebo na začátku skriptu `PATH=/usr/local/bin:/usr/bin:/bin`.
- **% v cronu** se interpretuje jako newline – escapovat na `\%` nebo volat ze skriptu.

- **TZ:** cron běží v systémové timezone; pro fixní timezone přidat `CRON_TZ=Europe/Prague` na začátek crontabu.
- **MAILTO:** cron defaultně posílá stderr emailem rootovi. Bud' `MAILTO=""` (vypnout), nebo nastavit `MAILTO=ops@firma.cz`.

18.4.3. systemd timer

Robustnější varianta pro produkční servery: lepší logy (`journalctl`), nativní retry, závislosti, jednotky lze stage-ovat.

Service unit (`/etc/systemd/system/claude-digest.service`):

[Unit]

Description=Claude weekly commit digest

Wants=network-online.target

After=network-online.target

[Service]

Type=oneshot

User=claude

Group=claude

WorkingDirectory=/opt/projects/api

EnvironmentFile=/etc/claude/env # ANTHROPIC_API_KEY=...

ExecStart=/usr/local/bin/claude -p "Vygeneruj týdenní digest commitů..." \
 --permission-mode plan \
 --max-turns 12 \
 --output-format json \
 --strict-mcp-config /etc/claude/mcp-prod.json

StandardOutput=append:/var/log/claude/digest.out

StandardError=append:/var/log/claude/digest.err

Retry: max 3 pokusy s odstupem 60 s

Restart=on-failure

RestartSec=60

StartLimitBurst=3

StartLimitIntervalSec=600

Sandboxing systemd

NoNewPrivileges=true

PrivateTmp=true

ProtectSystem=strict

ProtectHome=true

ReadWritePaths=/opt/projects/api /var/log/claude

Timer unit (/etc/systemd/system/claude-digest.timer):

[Unit]

Description=Spouští claude-digest.service pondělí v 07:00

[Timer]

OnCalendar=Mon 07:00 Europe/Prague

Persistent=true

spustí i po výpadku

AccuracySec=1min

RandomizedDelaySec=5min

vyhne se thundering herd

[Install]

WantedBy=timers.target

Aktivace:

```
sudo systemctl daemon-reload
sudo systemctl enable --now claude-digest.timer
systemctl list-timers --all           # ověření
journalctl -u claude-digest.service -f  # živé logy
```

18.4.4. Secret management

Tabulka rozumných variant podle prostředí:

Prostředí	Doporučený mechanismus
Cron na osobním stroji	~/.config/claude/env s chmod 600, sourcovat ve wrapperu.
systemd timer na serveru	EnvironmentFile=s chmod 640 root:claude.
Více serverů	HashiCorp Vault / AWS Secrets Manager + sidecar, který vystaví env.
Kubernetes CronJob	secretRef na Secret resource.

🛡️ **Nikdy** nedávat `ANTHROPIC_API_KEY` přímo do crontabu – `/var/spool/cron/crontabs/*` je čitelné rootem i zálohovacími skripty.

18.4.5. Logování a rotace

- Cron: výstupy do `/var/log/claude/`, rotace přes `logrotate` (`/etc/logrotate.d/claude`):

```
/var/log/claude/*.log /var/log/claude/*.json /var/log/claude/*.err {
    weekly
```



```

    rotate 8
    compress
    missingok
    notifempty
    create 640 claude claude
}

```

- systemd: `journalctl -u claude-digest.service --since "1 week ago"` . Rotace journalu se řídí `/etc/systemd/journald.conf` (`SystemMaxUse=`).
- `--output-format json` umožní strojové zpracování (jq, Loki, Elastic).

18.4.6. Idempotence a retry

Headless `claude -p` může selhat z externích důvodů (rate limit, timeout MCP serveru, síť). Doporučené vzory:

- **Lock soubor** (`flock`) – zabrání souběhu, pokud předchází běh ještě nedoběhl (typicky u long-running úloh, kde interval < trvání).
- **Idempotentní prompt** – formulujte úkol tak, aby opakované spuštění neudělalo škodu („pokud digest pro tento týden už existuje, neposílej znova“). Hooks (§ 14) mohou tuto kontrolu vynutit.
- **Retry s backoffem** – ve wrapper skriptu nebo přes `Restart=on-failure` + `RestartSec` v systemd.
- **Max-turns** – `--max-turns N` jako pojistka proti smyčkám v dlouho běžících úkolech.
- **Permission mode** – pro pravidelné runny preferujte `plan` (read-only) nebo `acceptEdits` s úzkým `--add-dir` ; nikdy `--dangerously-skip-permissions` mimo ephemeral kontejner.

18.4.7. Antipatterny

- × Cron volá `claude` bez wrapperu, bez logování, bez locku → tichá selhání.
- × API klíč v crontabu nebo v `git` -trackovaném skriptu.
- × Bez `--max-turns` → agent může uvíznout a generovat tokeny do nekonečna.
- × Bez `--strict-mcp-config` → produkční job vidí experimentální MCP servery.
- × Spouštění v `acceptEdits` nad sdíleným repem bez locku → race condition na souborech.

18.5. Pipeline vzor: vícefázový job

```

#!/usr/bin/env bash
set -euo pipefail

PLAN=$(claude -p "Naplánuj implementaci feature/oauth" \
  --permission-mode plan \
  --output-format json | jq -r '.result')

echo "$PLAN" > plan.md

```

```
claude -p "Implementuj plán z plan.md" \  
  --permission-mode acceptEdits \  
  --max-turns 30 \  
  --output-format json > impl.json
```

```
claude -p "Spust testy a oprav padající" \  
  --permission-mode acceptEdits \  
  --output-format json
```

18.6. Debugging headless

- `--debug` – vypíše system prompt, paměť, načtené nástroje.
- `--mcp-debug` – odstraní záhady kolem MCP.
- `2> claude.err` – zachytí chyby.

19. Stream-JSON formát

Stream-JSON je NDJSON – jeden JSON objekt na řádek. Slouží k živému zpracování průběhu odpovědi.

19.1. Output: `--output-format stream-json --verbose`

```
{"type":"system","subtype":"init","session_id":"...","model":"claude-opus-4-6"}  
{"type":"stream_event","event":{"type":"turn_start","turn":1}}  
{"type":"stream_event","event":{"type":"message_start"}}  
{"type":"stream_event","event":{"type":"message_delta","delta":  
{"type":"text_delta","text":"Začnu tím, že..."}}}  
{"type":"stream_event","event":{"type":"tool_call","tool":"Read","input":  
{"path":"/repo/Program.cs"}}}  
{"type":"stream_event","event":{"type":"tool_result","output":"using  
System;\n..."}}  
{"type":"stream_event","event":{"type":"message_stop"}}  
{"type":"final_result","cost":  
{"input_tokens":4231,"output_tokens":908,"cost_usd":0.041}}
```

19.2. Input: `--input-format stream-json`

Umožňuje injektovat víc uživatelských zpráv bez restartu CLI:

```
{"type":"user","message":{"role":"user","content":  
[{"type":"text","text":"Vysvětli soubor README"}]}}
```

```
{"type":"user","message":{"role":"user","content":[{"type":"text","text":"A  
teď napiš krátké shrnutí v 3 bodech"}]}}
```

```
cat prompts.jsonl | claude -p \  
  --input-format stream-json \  
  --output-format stream-json \  
  --verbose
```

19.3. Příklady zpracování

Extrakce delta tokenů přes **jq** :

```
claude -p "..." --output-format stream-json --verbose \  
  | jq -r 'select(.event?.delta?.text?) | .event.delta.text'
```

Sledování tool calls:

```
claude -p "..." --output-format stream-json --verbose \  
  | jq -c 'select(.event.type=="tool_call") | {tool:.event.tool,  
  input:.event.input}'
```

Záznam ceny každé relace v CSV:

```
claude -p "..." --output-format stream-json --verbose \  
  | jq -r 'select(.type=="final_result") |  
  [.cost.input_tokens,.cost.output_tokens,.cost.cost_usd] | @csv' \  
  >> usage.csv
```

20. Output styles a vzhled

Output style je sada „módu komunikace“ Claudu při psaní odpovědí.

20.1. Vestavěné

Styl	Charakteristika
default	Obvyklý helper, vyvážený.
explanatory	Detailní vysvětlení každého kroku, vhodné pro učení.
learning	Aktivní pedagogika: kvíz otázky, dílčí cíle, kontroly porozumění.

20.2. Vlastní styly

Custom style je markdown s frontmatterem v `~/.claude/output-styles/<name>.md` :

```
---
name: senior-cs-reviewer
description: Stručné review v duchu seniorního C# vývojáře.
---
```

Odpovídej česky, technicky stručně, s důrazem na SOLID, performance a maintainability.

Když navrhuješ změny, vždy uveď benchmark očekávání a alternativu.

Aktivace v relaci: `/output-style senior-cs-reviewer` .

20.3. Změna tématu (barev)

`/theme` – výběr ANSI tématu (light, dark, vysoký kontrast, dyslexia-friendly).

21. Vim mode, klávesové zkratky a terminál

21.1. Vim mode

`/vim` zapne Vim editing pro vstupní pole.

- `Esc` – z INSERT do NORMAL.
- `i` , `a` , `o` – do INSERT.
- `dd` , `yy` , `p` , `u` , `:wq` – jak zvyklost.
- `Ctrl+[` (na macOS `Ctrl+Shift+[`) – toggle insert/normal.

21.2. Globální zkratky

Klávesa	Účinek
Shift+Tab	Cyklus permission modu (default → acceptEdits → plan).
Ctrl+R	Re-render obrazovky / hledání v historii.
Esc	Přerušit aktuální generaci.
Ctrl+C	Tvrdé ukončení relace.
Shift+Enter	Nový řádek bez odeslání (po <code>/terminal-setup</code>).
Ctrl+J	Alternativa k Shift+Enter.
@	File mention v promptu.

#	Přidat větu do paměti (interaktivní add to memory).
/	Otevřít slash menu.

21.3. /terminal-setup

Zapíše tyto keybindings do shell rc souboru:

- bash: ~/.bashrc
- zsh: ~/.zshrc
- PowerShell: \$PROFILE

⚠ Spouštějte ze své host shellu, ne uvnitř `tmux` / `screen` – jinak se zapíše do nesprávného souboru.

21.4. Vlastní keybindings

/keybindings otevře ~/.claude/keybindings.json :

```
{
  "bindings": [
    { "key": "ctrl+s", "command": "submit" },
    { "key": "ctrl+l", "command": "clear" },
    { "key": "ctrl+m", "command": "open-memory" }
  ]
}
```

22. Cost a usage tracking

22.1. Vestavěné nástroje

- /cost – aktuální cena relace (jen pro API uživatele; subscription tarify mají vlastní limit).
- /status – aktivní účet (Pro / Max / Team / API), zbývající limit, model.

22.2. JSON metadata v headless

```
claude -p "..." --output-format json | jq '.cost'
```

```
{
  "input_tokens": 4231,
  "output_tokens": 908,
  "cache_creation_tokens": 121,
  "cache_read_tokens": 4203,
```

```
"cost_usd": 0.041
}
```

22.3. Optimalizace nákladů

- `DISABLE_NON_ESSENTIAL_MODEL_CALLS=true` – vypne vedlejší volání modelu (titulky, summaries, autoassist).
- `defaultHaikuModel` na novější/levnější Haiku revizi.
- `cleanupPeriodDays=1` – kratší retence transkriptů (méně auto-summary calls při resume).
- Subagenti s `model: claude-haiku-4-5` na rutinní úkoly (lint, formatter checks).
- Plan mode → review → implementace v `acceptEdits` (méně iterací).
- Skills s precizním `description` – Claude netáhne irrelevantní skills.

23. Best practices

23.1. Mentální model

- Začínáte v **plan mode** (`Shift+Tab`). Nechte si schválit plán.
- **Mezi úkoly použijte `/clear`** – kontext se neudrží přes nesouvisející aktivity.
- **`/compact`** zachrání tokeny v dlouhém vlákně.

23.2. CLAUDE.md hygiena

- ≤ 200 řádek na soubor.
- Konvence projektu, build příkazy, „čeho se nedotýkat“.
- Aktualizujte, když se změní setup; ideálně ve stejné PR.

23.3. Práce s nástroji a oprávněními

- `permissions.deny` chrání před náhodným `rm -rf`, čtením `.env`, postupy přes `curl | sh`.
- `permissions.allow` urychlí workflow – nemusíte potvrzovat každý `git status`.
- `defaultMode: "default"` v project scope, `"acceptEdits"` jen v personal `settings.local.json`.

23.4. Skills, agenti, hooks

- Skills pro **opakovaná know-how**.
- Agenti pro **úkoly s vlastním kontextem** (např. „test writer“).
- Hooks pro **deterministické vynucení** (formátování, audit, prevence secrets).

23.5. Slash commands

- Reusable workflow (lint, scaffolding, code review) přesuňte do `.claude/commands/`.
- Pojmenujte krátce a srozumitelně – v `/help` se vejde víc.
- Používejte `argument-hint` pro UI hint.

23.6. Verzování konfigurace

- `.claude/settings.json` , `.claude/agents/` , `.claude/commands/` , `.claude/skills/` , `.mcp.json` **commitovat**.
- `.claude/settings.local.json` , `.claude/local/` přidat do `.gitignore` .
- `CLAUDE.md` **commitovat**, `CLAUDE.local.md` **ignorovat**.

23.7. Bezpečnostní rytmus

-  Žádné API klíče v `.json` souborech v repu.
-  MCP servery, které drží auth tokeny, dávejte do **user scope**.
-  Před `--dangerously-skip-permissions` se ujistěte, že běžíte v sandboxu.

23.8. Praktické pracovní vzory

Vzor: „Plánuj → implementuj → testuj“

```
> [plan mode aktivní] Navrhni postup migrace EF6 → EF Core 9 v projektu.
... [Claude vypíše plán] ...
> [Shift+Tab → acceptEdits] Aplikuj kroky 1–3 z plánu.
... [Claude provede změny] ...
> Spusť `dotnet test` a oprav padající testy.
```

Vzor: „Subagent na unit testy“

```
> Use the csharp-test-writer agent to add tests for OrderService and
OrderRepository.
```

Vzor: „Skill na release notes“

```
> /release-notes
```

(Skill `release-notes` přečte git log a vygeneruje notes do `CHANGELOG.md` .)

23.9. Antipatterny

- × „Udělej cokoli“ prompty bez kontextu – Claude se ztrácí, kontext nestačí.
- × Jeden agent na všechno – ztrácí specializaci.
- × `bypassPermissions` na hostu.
- × Tisíce řádek `CLAUDE.md` – zaplaví system prompt.
- × Dlouhé jednorázové relace bez `/clear` .

24. Troubleshooting

24.1. „Tool denied by permissions“

Příčina: pravidlo v `permissions.deny` nebo chybějící v `permissions.allow`.

Řešení:

1. `/permissions` – přidejte výjimku.
2. Případně `--allowed-tools` flag pro tuto relaci.
3. Zkontrolujte hierarchii (managed → local → project → user).

24.2. „Context limit exceeded“

Příčina: dlouhé vlákno, příliš velká `CLAUDE.md`, mnoho načtených skillů s velkým tělem.

Řešení:

- `/compact`.
- `/clear` + nově načtený kontext.
- Refaktor `CLAUDE.md` na menší kapitoly + import `@`.
- Zkontrolovat `/context` a najít největšího žrouta.

24.3. „MCP server failed to start“

Řešení:

- `claude --mcp-debug` pro raw RPC log.
- `claude doctor` – ověří instalaci závislostí.
- Pro stdio servery: ověřte, že `command` existuje (`which`) a má prováděcí práva.
- Pro SSE: zkontrolujte, že `Authorization` header je správně.

24.4. „Claude se neptá, ale měl by“

- Zkontrolujte `permissions.defaultMode` – možná `acceptEdits`.
- Zkontrolujte `permissions.allow` – příliš liberální pattern.

24.5. Aktualizace selhala

```
claude doctor
DISABLE_AUTOUPDATER=true claude
claude update
```

Pokud i to selže, nainstalujte z čistého stavu (viz oficiální docs).

24.6. Pomalý start

- `/context` – kontrola velikosti system promptu.
- `cleanupPeriodDays` snížit – méně transkriptů k procházení.
- Vypnout MCP servery, které neaktivně používáte.

24.7. Stream-JSON nevrací delta tokeny

Příčina: chybí `--verbose` . Bez `--verbose` se stream zužuje na `final_result` .

24.8. „Cannot read .env"

Patrně máte `Read(./env)` v `deny` . Pokud to úmyslně chcete povolit (lokální dev), přidejte `Read(./env)` do `allow` v `settings.local.json` . Nikdy v project `settings.json` .

24.9. Příkaz `claude --help` nezná flag, který tu vidím

`--help` není kompletní. Některé flagy (např. `--input-format` , `--permission-prompt-tool`) nejsou v krátké nápovědě. Zde je úplná reference v Příloze A.

Příloha A – Kompletní reference flagů

Flag	Hodnota	Význam
<code>-p, --print</code>	–	Headless one-shot.
<code>--output-format</code>	<code>text json stream-json</code>	Formát stdout.
<code>--input-format</code>	<code>text stream-json</code>	Formát stdin.
<code>--max-turns</code>	<code>int</code>	Limit turnů.
<code>--verbose</code>	–	Detailní log; nutné pro stream-json delta tokeny.
<code>--debug</code>	–	Diagnostický výpis.
<code>--mcp-debug</code>	–	Raw MCP JSON-RPC log.
<code>-c, --continue</code>	–	Načte poslední relaci.
<code>-r, --resume</code>	<code>UUID</code>	Obnoví relaci.
<code>--session-id</code>	<code>string</code>	Textová značka.
<code>--model</code>	<code>model id</code>	Vynucený model.
<code>--fallback-model</code>	<code>model id</code>	Záloha.
<code>--add-dir</code>	<code>path</code>	Přidání adresáře.
<code>-w, --worktree</code>	<code>branch</code>	Izolovaný git worktree.
<code>--allowed-tools</code>	<code>pattern list</code>	Whitelist nástrojů.
<code>--disallowed-tools</code>	<code>pattern list</code>	Blacklist nástrojů.
<code>--permission-mode</code>	<code>default/acceptEdits/plan/bypassPermissions</code>	Mód oprávnění.

--permission-prompt-tool	mcp tool	Externí rozhodování.
--dangerously-skip-permissions	–	 Vypne kontroly.
--system-prompt	string	Plně přepsat system prompt.
--system-prompt-file	path	Plně přepsat ze souboru.
--append-system-prompt	string	Přidat na konec.
--append-system-prompt-file	path	Přidat ze souboru.
--mcp-config	json/path	Konfigurace MCP.
--strict-mcp-config	–	Pouze MCP z --mcp-config.
--settings, --config	path	Vynucený settings.json.
--ide	vscode/jetbrains	Cíl IDE.
--help	–	Krátká nápověda.
--version	–	Verze.

Příloha B – Reference klíčů

settings.json

Klíč	Typ	Účel
model	string	Výchozí model.
fallbackModel	string	Záložní model.
defaultHaikuModel	string	Konkrétní Haiku revize.
defaultSonnetModel	string	Konkrétní Sonnet revize.
defaultOpusModel	string	Konkrétní Opus revize.
cleanupPeriodDays	int	Retence transkriptů.
includeCoAuthoredBy	bool	Co-Authored-By v commitech.
autoMemoryEnabled	bool	Automatické memo Claudu.
apiKeyHelper	path	Skript vracející API klíč.
permissions.defaultMode	string	default/acceptEdits/plan/bypassPermissions.
permissions.allow	array	Whitelist patternů.

permissions.deny	array	Blacklist patternů.
permissions.ask	array	Nástroje, které vyžadují potvrzení.
permissions.additionalDirectories	array	Přidané adresáře.
env	object	Env proměnné injektované do session.
hooks	object	Hook konfigurace.
mcpServers	object	MCP konfigurace.

Příloha C – Reference environment proměnných

Proměnná	Účel
ANTHROPIC_API_KEY	API klíč.
ANTHROPIC_AUTH_TOKEN	Bearer token.
ANTHROPIC_BASE_URL	Vlastní endpoint.
ANTHROPIC_CUSTOM_HEADERS	JSON s hlavičkami.
ANTHROPIC_MODEL	Globální výchozí model.
ANTHROPIC_DEFAULT_HAIKU_MODEL	Haiku model.
ANTHROPIC_DEFAULT_SONNET_MODEL	Sonnet model.
ANTHROPIC_DEFAULT_OPUS_MODEL	Opus model.
CLAUDE_CODE_USE_BEDROCK	AWS Bedrock.
CLAUDE_CODE_USE_VERTEX	Google Vertex.
CLAUDE_CODE_SKIP_BEDROCK_AUTH	Přeskočit Bedrock auth.
AWS_REGION	AWS region.
CLOUD_ML_REGION	Google region.
ANTHROPIC_VERTEX_PROJECT_ID	Vertex projekt.
BASH_DEFAULT_TIMEOUT_MS	Default timeout pro Bash.
BASH_MAX_TIMEOUT_MS	Max timeout pro Bash.
BASH_MAX_OUTPUT_LENGTH	Max délka výstupu.
CLAUDE_CODE_MAX_OUTPUT_TOKENS	Limit output tokenů.
MCP_TIMEOUT	Timeout startu MCP.

MCP_TOOL_TIMEOUT	Timeout MCP tool callu.
MAX_THINKING_TOKENS	Limit thinking.
DISABLE_AUTOUPDATER	Vypnutí auto-update.
DISABLE_TELEMETRY	Vypnutí telemetrie.
CLAUDE_CODE_DISABLE_TELEMETRY	Alias.
DISABLE_BUG_COMMAND	Skrytí /bug.
DISABLE_COST_WARNINGS	Skrytí varování o ceně.
DISABLE_ERROR_REPORTING	Vypnutí pádových reportů.
DISABLE_NON_ESSENTIAL_MODEL_CALLS	Méně volání modelu.
DISABLE_PROMPT_CACHING	Vypnutí prompt cache.
NO_COLOR	Vypnutí ANSI barev.

Příloha D – Frontmatter cheat sheet

Custom command (`.claude/commands/<name>.md`)

```
---
description: Co command dělá (povinné).
allowed-tools: Bash(npm run *), Read, Edit
disallowed-tools: WebFetch
argument-hint: [arg1] [arg2]
model: inherit | haiku | sonnet | opus | claude-...
---
```

Subagent (`.claude/agents/<name>.md`)

```
---
name: short-id
description: Co agent dělá + kdy ho volat.
tools:
  - Read
  - Edit
disallowedTools:
  - WebFetch
model: claude-sonnet-4-6
```

```

permissionMode: default | acceptEdits | plan | bypassPermissions
maxTurns: 30
skills:
  - skill-name
memory: ./AGENT_MEMORY.md
isolation: worktree
effort: low | medium | high
color: blue
---
```

skill (`./.claude/skills/<name>/SKILL.md`)

```

---
```

```

name: skill-id
description: Kdy / na co se používá.
required:
  - Read
optional:
  - WebSearch
model: inherit | haiku | sonnet | opus
---
```

Output style (`~/.claude/output-styles/<name>.md`)

```

---
```

```

name: style-id
description: Jaký tón / styl odpovědí.
---
```

Příloha E – Glossář pojmů

Pojem	Význam
Agent	„Mini-Claude“ s vlastním system promptem a sadou nástrojů.
Allowlist / Denylist	Whitelist / blacklist patternů pro nástroje.
CLAUDE.md	Markdown soubor s pamětí pro Claude Code.
Devcontainer	Docker / VM kontejner pro izolovaný vývoj.
Headless mode	Neinteraktivní spuštění (<code>claude -p</code>).

Hook	Shell skript napojený na životní cyklus.
MCP	Model Context Protocol – standard pro připojení nástrojů a zdrojů dat.
Permission mode	Režim, jak Claude potvrzuje akce.
Plan mode	Read-only režim pro plánování bez úprav.
Plugin	Distribuovatelný balíček (skills + agenti + commands + hooks + MCP).
Progressive disclosure	Postupné odhalování instrukcí (frontmatter → tělo → references).
Session	Jedna interaktivní relace s vlastním kontextem a UUID.
Skill	Instructional package s autodiskoverem podle popisu.
Slash command	Příkaz spouštěný v relaci přes /.
Stream-JSON	NDJSON formát pro streamovaný I/O v headless.
Subagent	Synonymum pro agent ve smyslu „delegovaný úkol“.
Worktree	Git mechanismus pro paralelní pracovní kopie větví.

Příloha F – Užitečné zdroje a odkazy

- Oficiální dokumentace: <https://docs.claude.com> (Claude Code sekce)
- CLI reference: <https://docs.claude.com/en/docs/claude-code/cli-reference>
- Slash commands: <https://docs.claude.com/en/docs/claude-code/slash-commands>
- Settings: <https://docs.claude.com/en/docs/claude-code/settings>
- Hooks: <https://docs.claude.com/en/docs/claude-code/hooks-guide>
- MCP: <https://docs.claude.com/en/docs/claude-code/mcp>
- Subagents: <https://docs.claude.com/en/docs/claude-code/sub-agents>
- Skills: <https://docs.claude.com/en/docs/claude-code/skills>
- Permissions: <https://docs.claude.com/en/docs/claude-code/permissions>
- Headless: <https://docs.claude.com/en/docs/claude-code/headless>
- Worktrees: <https://docs.claude.com/en/docs/claude-code/worktrees>
- Memory: <https://docs.claude.com/en/docs/claude-code/memory>
- Best practices: <https://docs.claude.com/en/docs/claude-code/best-practices>
- Environment variables: <https://docs.claude.com/en/docs/claude-code/env-vars>
- Tools reference: <https://docs.claude.com/en/docs/claude-code/tools-reference>

Konec dokumentu.